

16 Special Effects

We have seen a great many different methods of changing the appearance of an image, and usually our need for the particular change was motivated by a problem: we may need to remove something from an image (noise, for example); we may need to make the image “look better”; we may need to calculate some aspect of the image (size, position, number of components, etc.)

But there is another totally different aspect of image processing, that of adding some sort of special effect to an image. We may want to do this for effect alone, or simply for fun. There are a great many different effects possible, and in this chapter we shall look at a few of them. This is a sort of gray area of image processing; we are encroaching toward the area of computer graphics. However, our algorithms involve the changing either of the value or the position of pixels, and we can achieve many effects with very simple means.

16.1 Polar Coordinates

Many effects have a “radial” nature, their appearance “radiates” outward from the center of the image. In order to achieve these effects, we need to transform our image from Cartesian coordinates to polar coordinates, and back again.

The origin. Suppose we have an image of size `rows` x `cols`. If the dimensions are odd, we can choose as our origin the very center pixel. If the dimensions are even, we choose as origin a pixel at the top left of the bottom right quadrant. so in a 7×9 image, our origin would be at pixel (4, 5), and in a 6×8 image, our origin would be at position (4, 4). We can express the origin formally by

$$\begin{aligned}x_0 &= \lceil (r + 1)/2 \rceil \\y_0 &= \lceil (c + 1)/2 \rceil\end{aligned}$$

where r and c are the numbers of rows and columns, respectively, and $\lceil x \rceil$ is the *ceiling function*, which returns the smallest integer not less than x . Of course, if one dimension is odd and the other even, the origin will be at the center of the odd dimension, and in the first place of the second half of the even dimension.

Thus, we can find coordinates of the origin by either

```
>> ox = ceil((rows+1)/2)
>> oy = ceil((cols+1)/2)
```

MATLAB/Octave

or

```
In : ox = (rows+1)//2
In : oy = (cols+1)//2
```

Python

Polar coordinates. To find the polar coordinates, the usual formulas can be used:

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \tan^{-1}(y/x),$$

which can be implemented for an array by using the `meshgrid` function from MATLAB and Octave, or the `mgrid` function of Python, where the indices are offset by the values of the origin:

```
>> [y,x] = meshgrid([1:cols]-oy,[1:rows]-ox);
>> r = sqrt(x.^2+y.^2);
>> theta = atan2(y,x);
```

MATLAB/Octave

```
In : y,x = np.mgrid[-oy:cols-oy,-ox:rows-ox]
In : r = np.sqrt(x**2+y**2)
In : theta = np.arctan2(y,x)
```

Python

We can easily go back, using the fact that the (x, y) coordinates corresponding to the polar coordinates (r, θ) are

$$x = r \cos \theta$$

$$y = r \sin \theta.$$

Thus:

```
>> x2 = round(r.*cos(theta))+ox
>> y2 = round(r.*sin(theta))+oy
```

MATLAB/Octave

or

```
In : x2 = np.round(r*np.sin(theta))+ox
In : y2 = np.round(r*np.cos(theta))+oy
```

Python

and these arrays will contain the same indices as the original arrays.

It will be convenient to have a `polarmesh` function which provides r and θ values for a given array size, as well as a `polar2im` function which applies polar values to an image. Such functions are given at the end of the chapter.

An Example: Radial Pixelization

We have seen in Chapter 3 that an image can be “pixelated”; that is, be shown in large blocks of low resolution, by use of the `imresize` function. We can achieve the same effect, however, by using the `mod`

function. In general, $\text{mod}(x, n)$ is the remainder when x is divided by n . For example:

```
>> x=1:12
x =
     1     2     3     4     5     6     7     8     9    10    11    12

>> mod(x,4)
ans =
     1     2     3     0     1     2     3     0     1     2     3     0
```

MATLAB/Octave

In Python, the % operator provides modulus functionality:

```
In : x = np.array(range(1,13))
In : print np.vstack((x,x%4))
[[ 1  2  3  4  5  6  7  8  9 10 11 12]
 [ 1  2  3  0  1  2  3  0  1  2  3  0]]
```

Python

So if x is divisible by 4, then there is no remainder, and so the modulus function returns 0. Now if we *subtract* the mod values from the original, we obtain repetition, which in an image would produce a pixelated effect:

```
>> x-mod(x,4)
ans =
     0     0     0     4     4     4     4     8     8     8     8    12
```

MATLAB/Octave

```
print x-x%4
[ 0  0  0  4  4  4  4  8  8  8  8 12]
```

Python

So a subtraction of moduli from the radius and argument values should produce a radial pixelated effect.

We can try it with the flowers image, which for simplicity we shall turn into a grayscale image.

```
>> f = imread('iris.png');
>> fg = rgb2gray(f);
>> [r,theta] = polarmesh(fg);
```

MATLAB/Octave

In Python, the commands would be

```
In : f = io.imread('iris.png')
In : fg = co.rgb2gray(f)
In : r,theta = polarmesh(fg)
```

Python

At this stage we have the polar values of the mesh, so now we can perform the pixelation:

```
>> r2 = r-mod(r,5);
>> theta2 = theta-mod(theta,5*pi/180);
```

MATLAB/Octave

or

```
In : r2 = r - r%5
In : theta2 = theta - theta%(5*np.pi/180)
```

Python

Since the angles are given as radians, the modulus is equivalent to 5 degrees.

Now we transfer back into Cartesian coordinates:

```
>> x2 = r2.*cos(theta2);
>> y2 = r2.*sin(theta2);
```

MATLAB/Octave

or

```
In : x2 = r2*np.cos(theta2)
In : y2 = r2*np.sin(theta2)
```

Python

To obtain an image for display, we have to adjust the indices `x2` and `y2` by adding `ox` and `oy` to them, rounding off, and making sure that the row indices are between 1 and `rows`, and that the column indices are between 1 and `cols`. This last clipping will be made easier in MATLAB or Octave by a very small function:

```
function out = clip(x,y,z)
    out = x;
    out(find(out<y))=y;
    out(find(out>z))=z;
```

MATLAB/Octave

Now for the adjustment:

```
>> xx = round(x2)+ox;
>> yy = round(y2)+oy;
>> xx = clip(xx,1,rows);
>> yy = clip(yy,1,cols);
```

MATLAB/Octave

In Python again a clipping function will be useful:

```
In : def clip(x,y,z):
...:     x[np.where(x<y)]=y
...:     x[np.where(x>z)]=z
...:     return x
...:
```

Python

and the adjustment is very similar to that in MATLAB/Octave:

```
In : xx = np.round(x2)+ox
In : yy = np.round(y2)+oy
In : xx = clip(xx,0,rows-1).astype(int)
In : yy = clip(yy,0,cols-1).astype(int)
```

Python

Now we can use these new indices `xx` and `yy` to obtain an image from the original flower image matrix `fg`. This can be most simply done by using the `sub2ind` function, which basically finds places in an array indexed by given vectors.

```
>> f2 = fg(sub2ind([rows,cols],xx,yy));
```

MATLAB/Octave

In Python the `ravel` method produces one long vector of the array; the `xx` and `yy` vectors can be used as indices into `f`, the result of which is reshaped for display:

```
In : f2 = np.reshape(f[xx.ravel(),yy.ravel()],[rows,cols])
```

Python

And in fact all the last commands can be bundled into `polar2im` functions, which are given at the end of the chapter, so that

```
In : f2 = polar2im(im,r2,theta2);
```

MATLAB/Octave

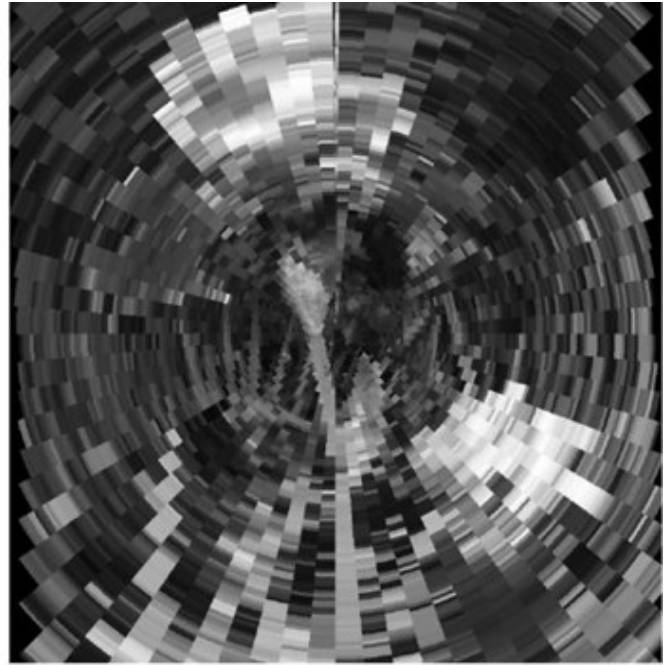
(and the same command in Python) is all that is needed.

The initial image and the radiated pixelated result are shown in Figure 16.1.

Figure 16.1: Radial pixelation



The original *fg*



The pixelated *f2*

16.2 Ripple Effects

We will investigate two different ripple effects: “bathroom glass” ripples, which give the effect of an image seen through wavy glass, such as is found in bathrooms, and “pond ripples,” which approximate a reflection in the surface of a pond.

We have seen that subtracting moduli produces a pixelated effect. A bathroom glass effect can be obtained by *adding* moduli:

```
>> x=1:12;  
>> x+mod(x,4)  
ans =  
     2     4     6     4     6     8    10     8    10    12    14    12
```

MATLAB/Octave

or

```
In : x = np.array(range(1,13))  
In : print np.vstack((x,x+x%4))  
[[ 1  2  3  4  5  6  7  8  9 10 11 12]  
 [ 2  4  6  4  6  8 10  8 10 12 14 12]]
```

Python

Notice that although the values increase from left to right, they do in small runs, which seem to overlap. On an image, the appearance will be of a ripple across the image. We shall experiment first using Cartesian coordinates:

```
>> [y,x] = meshgrid(1:cols,1:rows);  
>> y2 = y+mod(y,32);  
>> y2 = clip(y2,1,cols);
```

MATLAB/Octave

Here we have just added moduli across the columns, then adjusted the result to ensure that the values stay within the column range. Now we can use `sub2ind` to create a new image:

```
>> ripple1 = fg(sub2ind([rows cols],x,y2));  
>> imshow(ripple1)
```

MATLAB/Octave

We can of course do the same thing down the rows:

```
>> x2 = x+mod(x,32);  
>> x2 = clip(x2,1,rows);  
>> ripple2 = fg(sub2ind([rows cols],x2,y));  
>> imshow(ripple2)
```

MATLAB/Octave

or even both at once:

```
>> ripple3 = fg(sub2ind([rows cols],x2,y2));  
>> imshow(ripple3)
```

MATLAB/Octave

In Python, the vertical ripples can be obtained with

```
In : y,x = np.mgrid[0:cols,0:rows]  
In : x2 = clip(x+x%32,0,rows-1)  
In : ripple1 = np.reshape(fg[x2.ravel(),y.ravel()],(rows,cols)).T
```

Python

horizontal ripples with

```
In : y2 = clip(y+y%32,0,cols-1)  
In : ripple2 = np.reshape(fg[x.ravel(),y2.ravel()],(rows,cols)).T
```

Python

and both with

```
In : ripple3 = np.reshape(fg[x2.ravel(),y2.ravel()],[rows,cols]).T
```

Python

To obtain a radial ripple, corresponding to ripples on a pond, we create the polar coordinate matrices `r` and `theta` with `polarmesh`, adjust the radii, and finally use `polar2im` to obtain the output image:

```
>> r2=r+mod(r,30);  
>> ripple4 = uint8(polar2im(fg,r2,theta));
```

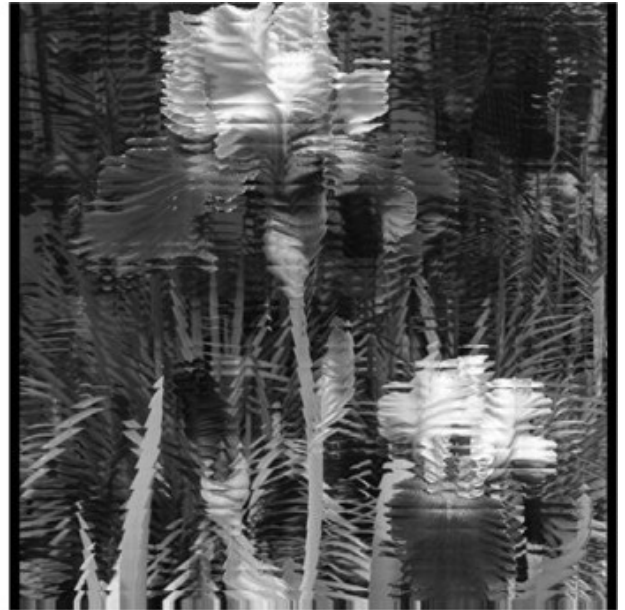
MATLAB/Octave

All results are shown in Figure 16.2.

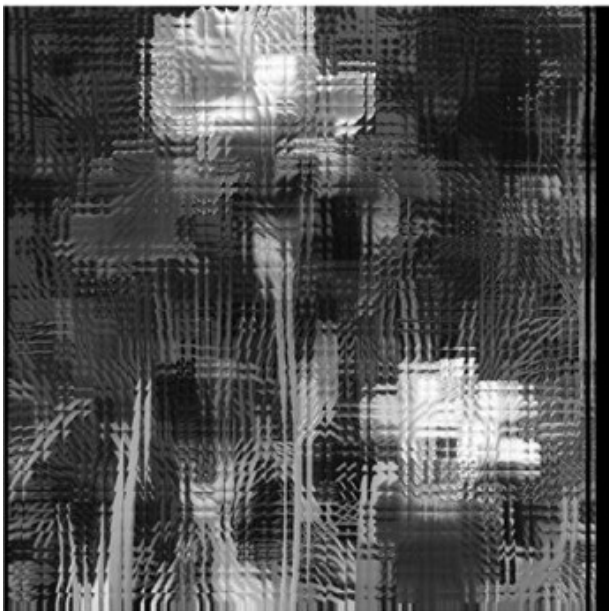
Figure 16.2: “Bathroom glass” rippling on an image



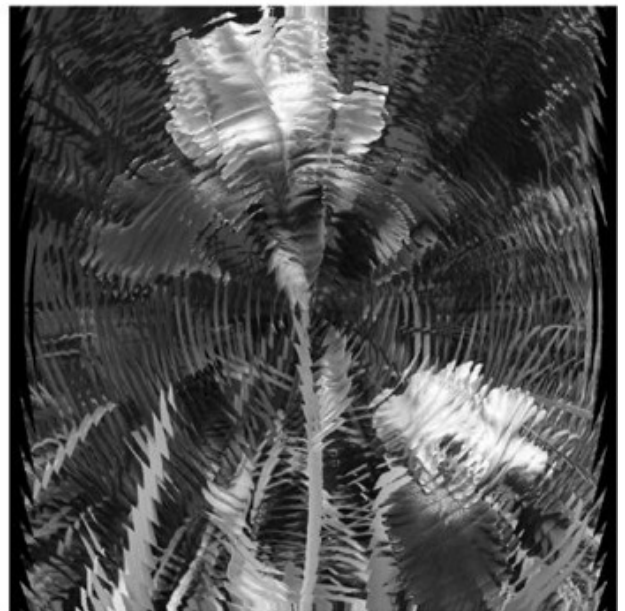
(a) ripple1



(b) ripple2



(c) ripple3



(d) Radial ripple: ripple4

To obtain a pond ripple effect, we move the pixels around by using a sine wave. For Cartesian effects:

```
>> [y,x] = meshgrid(1:cols,1:rows);  
>> y2 = clip(round(y+7*sin(x/5)),1,cols);  
>> x2 = clip(round(x+7*sin(y/5)),1,rows);
```

MATLAB/Octave

or with

```
In : y,x = np.mgrid[0:rows,0:cols]
In :
```

Python

As above, we ensure that the values of x_2 and y_2 fit within the row and column bounds. Then:

```
>> ripple5 = fg(sub2ind([rows cols],x,y2));
>> ripple6 = fg(sub2ind([rows cols],x2,y));
>> ripple7 = fg(sub2ind([rows cols],x2,y2));
```

MATLAB/Octave

For a radial sine wave ripple, we adjust r first:

```
>> r2 = r + 7*sin(r/5);
>> ripple8 = polar2im(fg,r2,theta);
```

MATLAB/Octave

After that we create x_2 and y_2 as above, and turn them back into indices xx and yy with the appropriate bounds. Then

```
>> ripple8 = fg(sub2ind([rows cols],xx,yy));
```

MATLAB/Octave

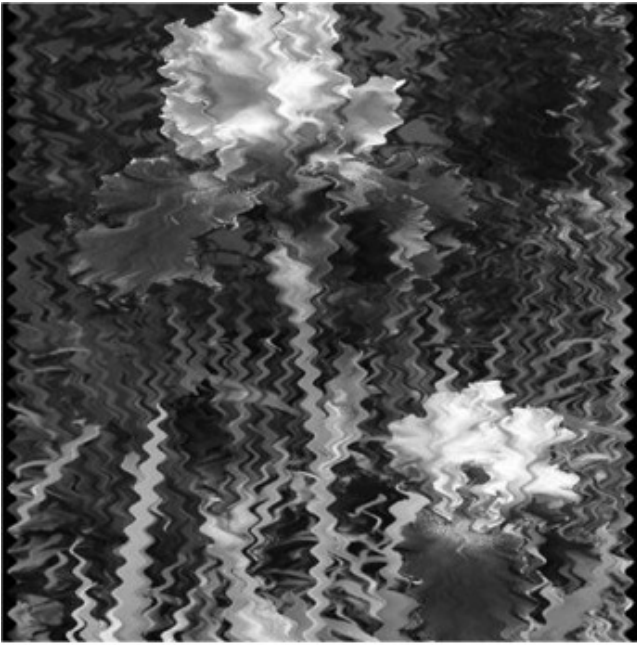
These new ripples are shown in Figure 16.3. The effects can be varied by changing the parameters to the sine functions in the commands above.

16.3 General Distortion Effects

The rippling effects in the previous section are examples of more general effects called *distortion effects*. Here it is not so much the *value* of the pixels that are changed as their *position*. All distortion effects work in much the same way, which we shall describe in a sequence of steps. Suppose we have an effect that involves Cartesian coordinates only.

1. Start with an image matrix $p(i, j)$, of size $m \times n$.
2. Produce two new indexing arrays (the same size as the matrix): $x(i, j)$ and $y(i, j)$.
3. Create a new image $q(i, j)$ by $q(i, j) = p(x(i, j), y(i, j))$.

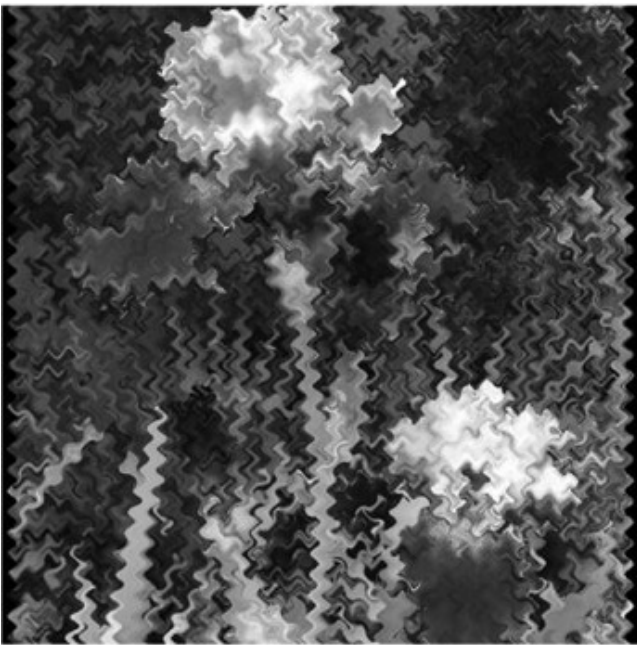
Figure 16.3: “Pond rippling” on an image



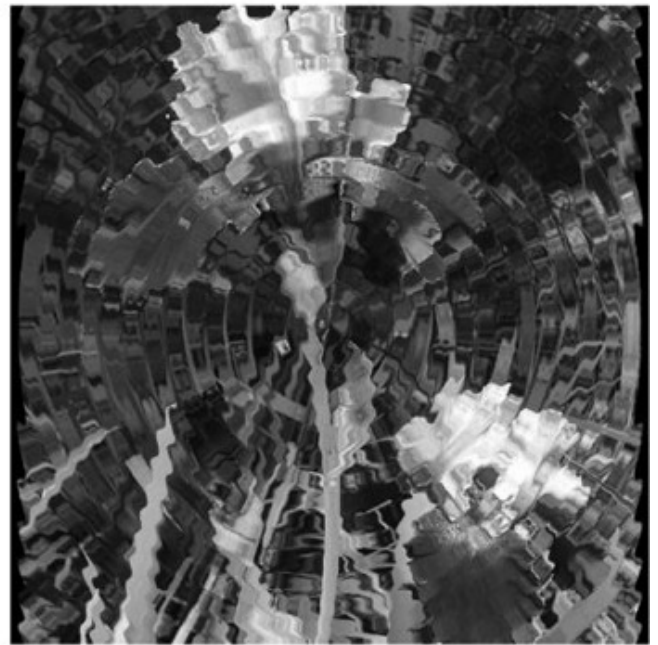
(a) ripple5



(b) ripple6



(c) ripple7



(d) ripple8

We have seen above that the arrays $x(i, j)$ and $y(i, j)$ may be created by a procedure that produces values which may be fractional or be outside the bounds of the image matrix. So we will need to ensure that the indexing arrays contain integers only, and that for all i and j , that $1 \leq x(i, j) \leq m$, and $1 \leq y(i, j) \leq n$.

If we have an effect that requires polar coordinates, we add a few steps to the procedure above:

1. Start with an image matrix $p(i, j)$, of size $m \times n$.
2. Create the arrays $r(i, j)$ and $\theta(i, j)$ containing the polar coordinates of each position (i, j) .
3. Produce new arrays $s(i, j)$ and $\varphi(i, j)$ by adjusting the values of r and θ .

4. Write these new arrays out to Cartesian indexing arrays (the same size as the matrix):
 $x(i, j) = s(i, j) \cos(\phi(i, j)) + o_x$
 $y(i, j) = s(i, j) \sin(\phi(i, j)) + o_y$ where (o_x, o_y) are the coordinates of the polar origin.
5. Create a new image $q(i, j)$ by $q(i, j) = p(x(i, j), y(i, j))$.

As above, we need to ensure that the indexing arrays contain only integers, and have the appropriate bounds.

We have seen above how to implement various distortion effects in MATLAB, so just by changing the functions used to distort the indexing arrays or the polar coordinates, we can achieve many different effects.

Fisheye. The “fisheye lens” effect can be obtained very easily with polar coordinates, with

$$r \leftarrow r^2/R$$

where

$$R = \max(r).$$

So with MATLAB and Octave, using the `polarmesh` and `polar2im` functions, the commands are:

```
>> [r,theta] = polarmesh(fg);
>> R = max(r(:));
>> s = r.^2/R;
>> f2 = uint8(polar2im(fg,s,theta));
```

MATLAB/Octave

And in Python they are very similar:

```
In : r,theta = polarmesh(fg)
In : s = r**2/r.max()
In : f2 = polar2im(fg,s,theta)
```

Python

The result is shown in Figure 16.4.

Twirl. A “twirl” or “swirl” effect, as the top of a cup of coffee being swirled with a spoon, is also very easily done. Again, it is a polar effect. A twirl effect can be obtained by

$$s(i, j) = \theta(i, j) + r(i, j)/K$$

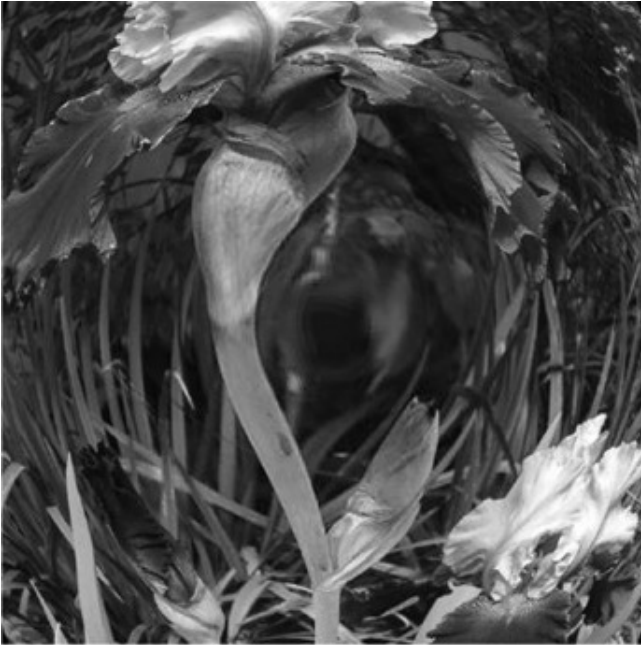
where K can be changed to adjust the amount of twirl. A small value provides more twirl, a large value less. So given the arrays `r` and `theta`, and a value `K`, the relevant commands are:

```
>> phi = theta+(r/K);  
>> twirl1 = polar2im(fg,r,phi);
```

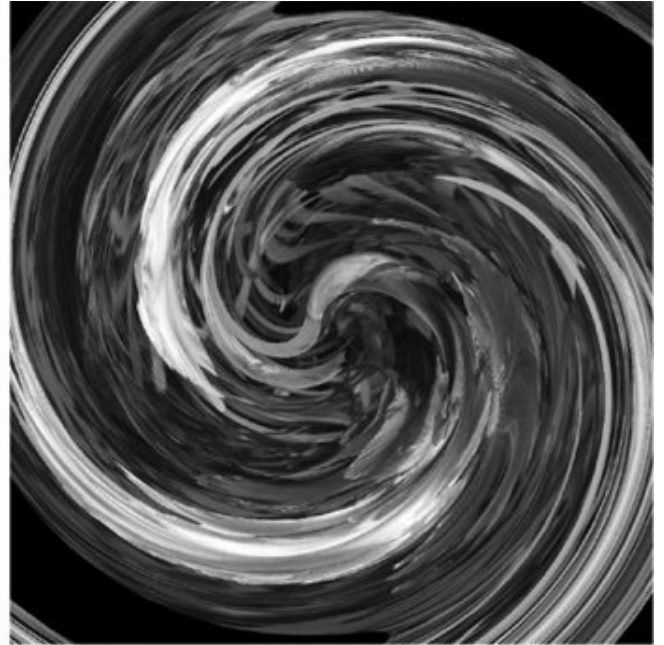
MATLAB/Octave

Figure 16.4 shows the results of twirls with different values of K .

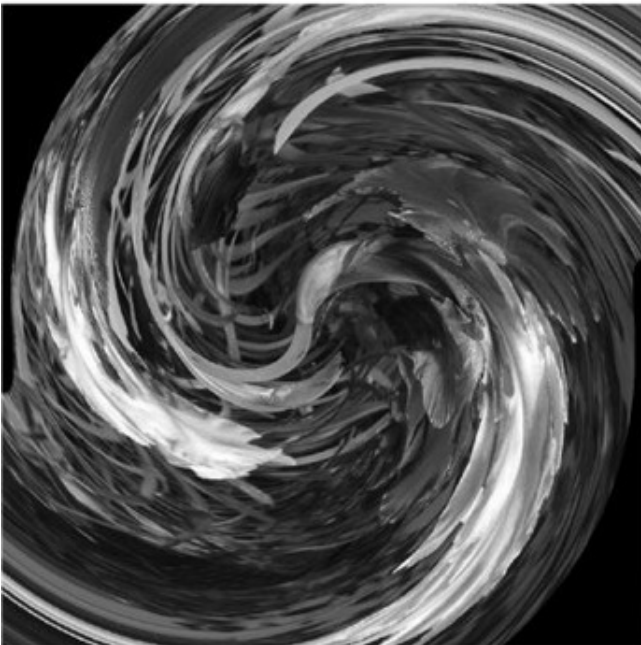
Figure 16.4: Fisheye and twirls



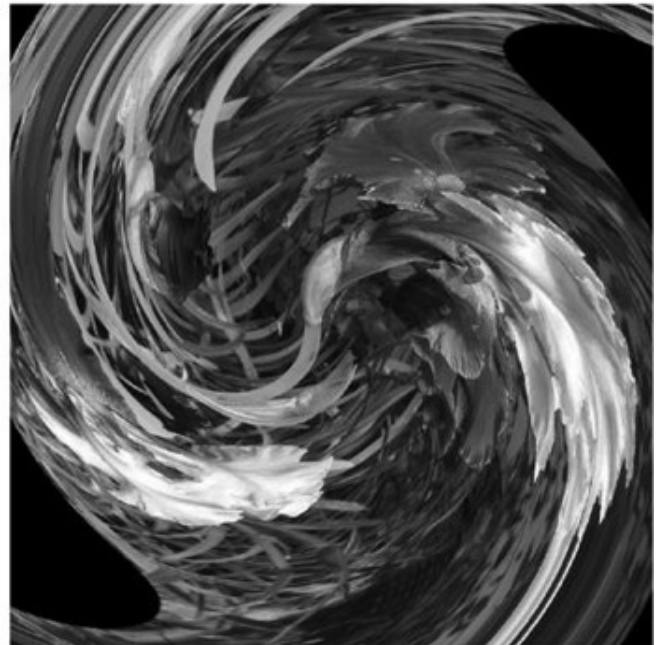
fisheye



A twirl with $K = 100$



A twirl with $K = 150$



A twirl with $K = 200$

Jitter. The “jitter” effect is more easily implemented than described. It is like a radial bathroom glass effect, but instead of manipulating the r matrix to obtain a ripple effect, we manipulate the θ matrix:

$$\phi = \theta + \text{mod}(\theta, 8\pi/180) - 4\pi/180.$$

```
>> phi = theta+mod(theta,8*pi/180)-4*pi/180;
```

MATLAB/Octave

```
In : phi = theta+(theta%(8*np.pi/180))-4*np.pi/180
```

Python

The result is shown in Figure 16.5.

Circular slice. This is another effect obtained by manipulating the angle. Its implementation is similar to that of the jitter effect:

$$\phi = \theta + \text{mod}(r, 6) * \pi/180$$

We include the extra $\pi/180$ factor to convert to radians. The result, which has a rippling appearance, is shown in Figure 16.5.

Figure 16.5: Jitter and circular slice



Jitter



Circular slice

Square slice This is a Cartesian effect. It works by cutting the image into small squares and perturbing them a little bit. It uses the “sign” function (sometimes called the *signum* function and abbreviated as *sgn*), which is given in MATLAB and Octave as `sign` and in Python as `np.sign`, and which is defined as:

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x = 0, \\ -1 & \text{if } x < 0. \end{cases}$$

We start with

```
>> [y,x] = meshgrid(1:cols,1:rows);
```

MATLAB/Octave

and then implement the slice distortion with

```
>> K=8;  
>> Q=10;  
>> x2=round(x+K*sign(cos(y/Q)));  
>> y2=round(y+K*sign(cos(x/Q)));
```

MATLAB/Octave

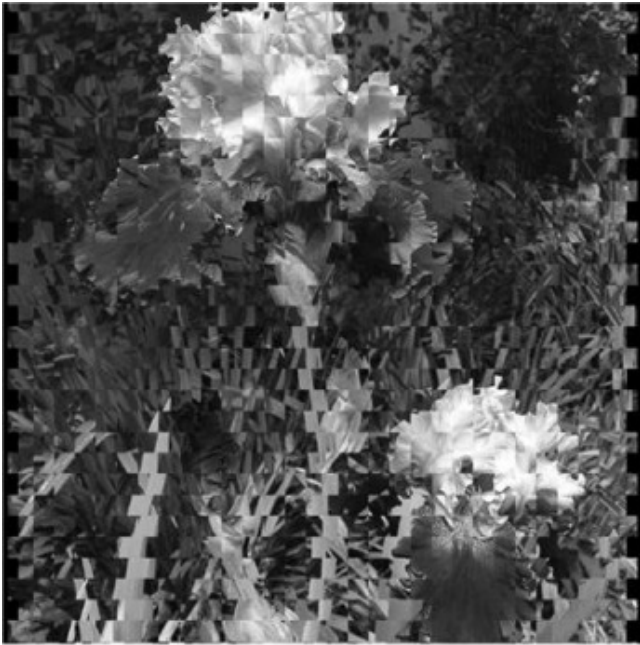
Then the final image can be computed with

```
>> f2 = fg(sub2ind(size(fg),clip(x2,1,rows),clip(y2,1,cols)));
```

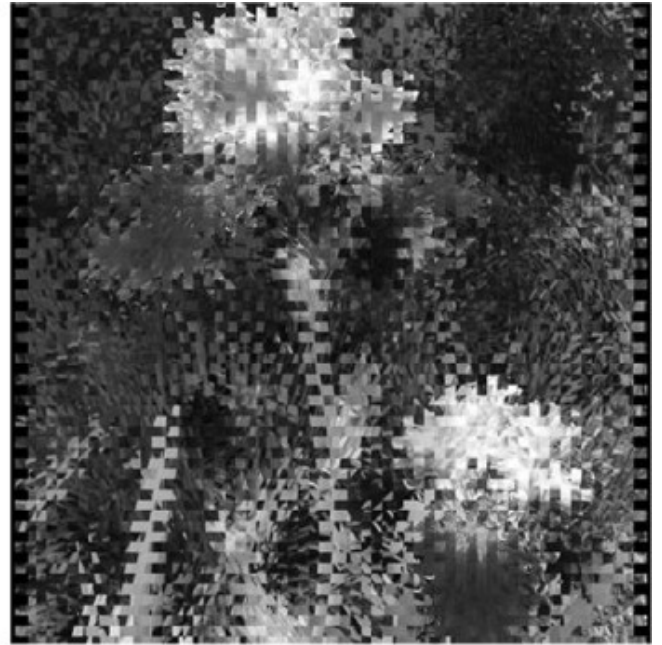
MATLAB/Octave

The values K and Q influence the size of the squares and the amount of perturbation. A large value of K produces a very perturbed image; a large value of Q produces large squares. After rounding, fixing the bounds and writing to a new image, results are shown in Figure 16.6.

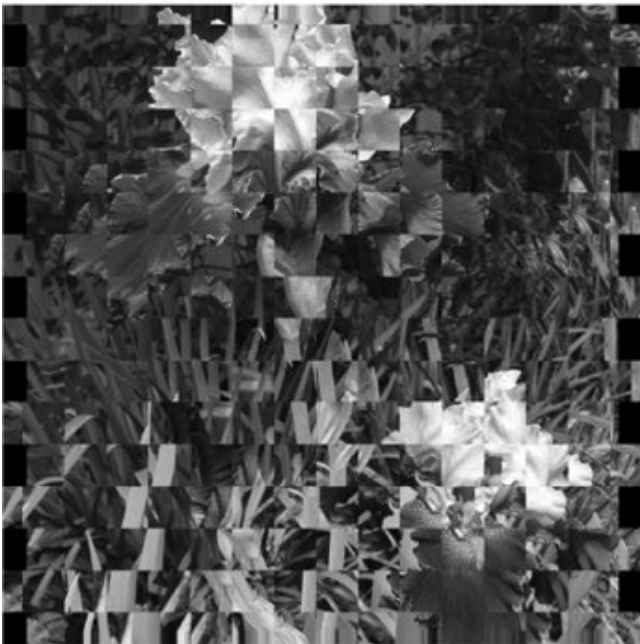
Figure 16.6: The “square slice” effect



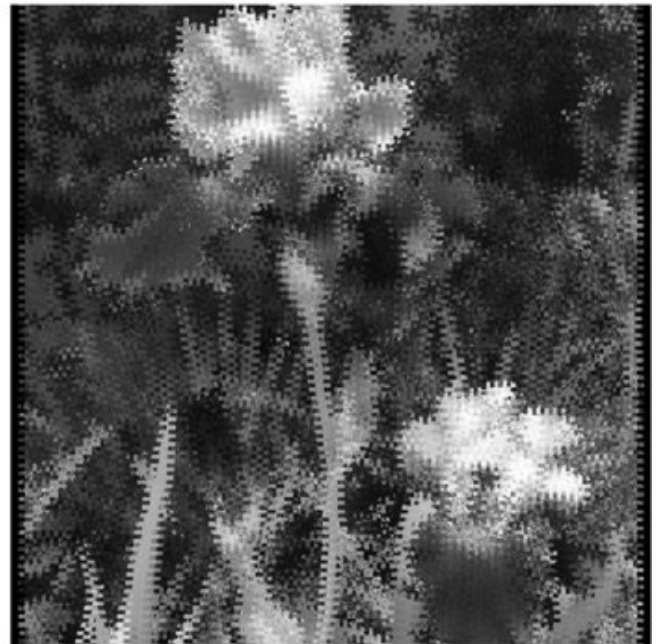
K=8;Q=10



K=10;Q=5



K=20;Q=10



K=10;Q=5

Fuzzy effect. This is obtained by replacing each pixel with a randomly selected pixel from its neighborhood. To do this, we simply add random values to the index matrices x and y . If we are choosing values from a 7×7 neighborhood, then we randomize the indices with integer values chosen from $-3, -2, -1, 0, 1, 2, 3$. We can achieve this in MATLAB or Octave with

```
>> x2 = x+randi([-3,3],rows,cols);  
>> y2 = y+randi([-3,3],rows,cols);
```


and in Python with

```
In : x2 = x + np.random.randint(-3,3,size=(rows,cols))  
In : y2 = y + np.random.randint(-3,3,size=(rows,cols))
```

Python

The MATLAB/Octave `randi` function given here produces a matrix of size `rows` x `cols` where each element is a random integer in the range given by the initial vector. To obtain the image:

```
>> f2 = fg(sub2ind(size(fg),clip(x2,1,rows),clip(y2,1,cols)));
```

MATLAB/Octave

or

```
In : xx = clip(x2,0,rows-1).astype(int)  
In : yy = clip(y2,0,cols-1).astype(int)  
In : f2 = np.reshape(f[xx.ravel(),yy.ravel()],(rows,cols)).T
```

Python

Results for different sized neighborhoods are shown in Figure 16.7. For a large neighborhood, the result begins to take on a frosted glass effect.

Figure 16.7: The fuzzy effect using different neighborhoods



5 × 5



7 × 7

16.4 Pixel Effects

In a sense, *all* image effects are “pixel effects,” because we are dealing with pixels. However, in the previous sections, we did not change the values of the grayscales explicitly; we merely copied them to different positions in the output arrays. However, many effects take the pixel values and apply some sort of processing routines to them.

Oil painting. This is a very popular effect, and is included in most image manipulation and photo handling software. In conception it is very simple; it works by means of a non-linear filter; the output of the filter is the most common pixel value in the filter mask.

The most commonly occurring value in a neighborhood is the statistical *mode*, and is implemented in MATLAB and Octave by the function `mode`. In Python, it is implemented by the `mode` function in the `scipy.stats` module. It can be applied to the iris image as:

```
>> f2 = colfilt(fg,[9,9], 'sliding', @mode);
```

MATLAB/Octave

with the `sliding` parameter indicating that the function is to be applied to each possible 9×9 neighborhood.

In Python, the `mode` function has to be adjusted slightly so as to return a single number, rather than both the number and its frequency:

```
In : def mymode(x):  
...:     return sc.stats.mode(x,axis=None)[0][0]  
...:  
In : f2 = ndi.generic_filter(fg,mymode,size=(9,9))
```

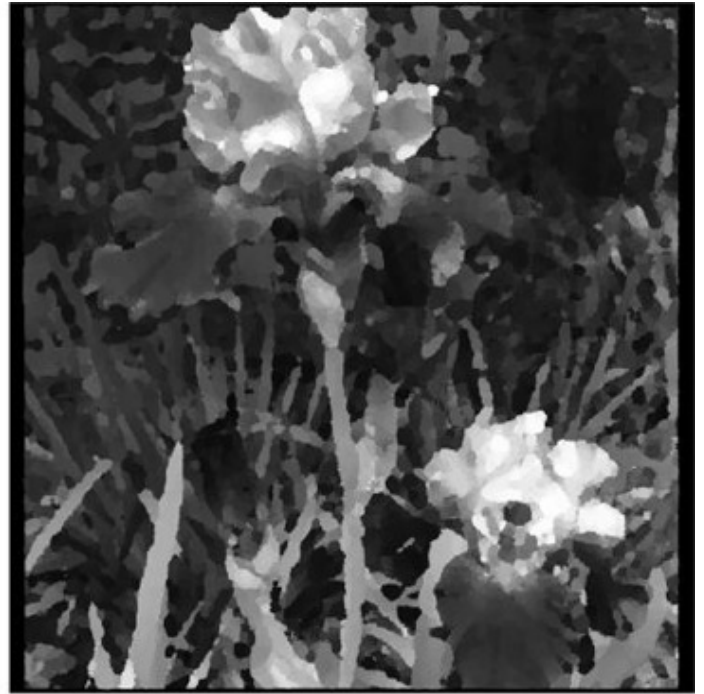
Python

The result is shown in Figure 16.8(a). We can obtain a more “painterly” effect by choosing a larger block size in `ndi.filter`. All of these functions can be slow!

Figure 16.8: The oil painting effect with different filter sizes



(a) 9×9



(b) 15×15

Solarization. This is a photographic effect that is obtained by applying diffuse light to a developing photograph, and then continuing with the development. The result, which seems somehow magical, is that the image can be partly positive and partly negative.¹ This effect has been used by many photographers, but can be a fairly hit or miss affair in the photographic darkroom. However, it is very easy to create solarization effects digitally.

We have seen examples of solarization functions in Chapter 4; now we explore how they can be applied.

A simple solarization effect can be obtained by taking the complement of all pixels in an image $p(i, j)$ whose grayscales are less than 128:

$$\text{sol}(i, j) = \begin{cases} p(i, j) & \text{if } p(i, j) > 128, \\ 255 - p(i, j) & \text{if } p(i, j) \leq 128. \end{cases}$$

This can be very easily implemented:

```
>> u = fg>128;  
>> sol = u.*fg + (1-u).*(255-fg);  
>> imshow(uint8(sol)), figure, imshow(mat2gray(sol))
```

MATLAB/Octave

We display the result twice; the second time the image is shown with better contrast.

In Python, the `rgb2gray` function returns a floating point array, so the image complement is obtained with `1 - fg`:

```
In : u = fg>0.5
In : sol = u*fg + (1-u)*(1-fg)
In : io.imshow(sol,vmin=0,vmax=1)
In : io.imshow(sol)
```

Python

Note that Python automatically adjusts the display so that the lowest element in the array is mapped to black and the largest element to white. To prevent that adjustment the `vmin` and `vmax` parameters must be used; these supply minimum and maximum values to `imshow`. Both images are shown in Figure 16.9. We can make more interesting solarization effects by varying the complementation according to the value of the row or column. All we need to do is to change the matrix `u` above:

Figure 16.9: The solarization effect



```
>> u1 = double(fg > 255*x/(2*rows));
>> u2=double(fg > 255*y/(2*cols));
```

MATLAB/Octave

or with

```
In : u1 = fg > np.float32(x)/(2.0*rows)
In : u2 = fg > np.float32(y)/(2.0*cols)
```

Python

The results are shown in Figure 16.10.

Figure 16.10: More solarization effects



sol1



sol2

1

A good account of solarization can be found at

<http://www.cchem.berkeley.edu/wljeme/SOUTLINE.html>.

16.5 Color Images

Most of the above effects look far more dramatic when applied to color images. For all the distortion effects, application to a color image means applying the effect to each of the RGB components separately. Recall that distortion effects do not change the values of the pixels so much as their position. Thus, in order to maintain the colors, we need to shift all the RGB components.

Here, for example, are twirl and circular ripple applied to the colored image, and assuming that all the index arrays x , y , r , and θ have already been computed:

```
>> f = imread('iris.png');  
>> K = 150;  
>> phi = theta + r/K;  
>> r2 = r + mod(r,30);  
>> twirl = zeros(size(f));  
>> ripple = zeros(size(f));  
>> for i = 1:3, twirl(:,:,i) = polar2im(f(:,:,i),r,phi); end  
>> for i = 1:3, ripple(:,:,i) = polar2im(f(:,:,i),r2,theta); end  
>> imshow(uint8(twirl)), figure, imshow(uint8(ripple))
```

MATLAB/Octave

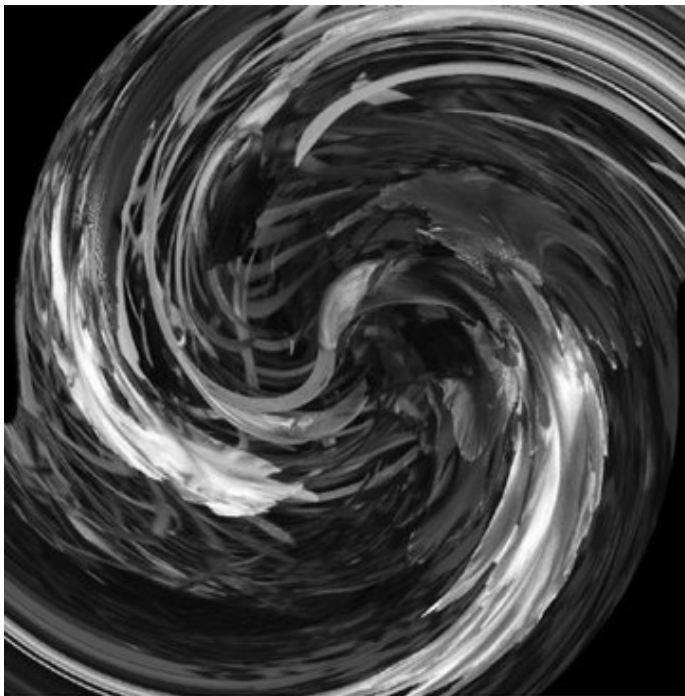
The method in Python is very similar:


```
In : f = io.imread('iris.png')
In : K = 150; phi = theta + r/K
In : r2 = r + (r%30)
In : twirl = np.zeros_like(f)
In : ripple = np.zeros_like(f)
In : for i in range(3):
...:     twirl[:, :, i] = polar2im(f[:, :, i], r, phi)
...:     ripple[:, :, i] = polar2im(f[:, :, i], r2, theta)
...:
```

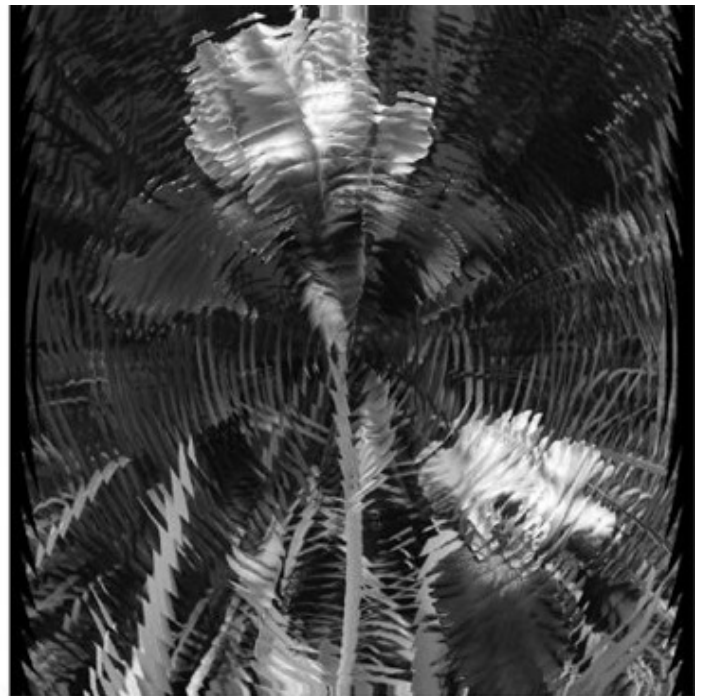
Python

The results are shown in Figure 16.11.

Figure 16.11: SEE COLOR INSERT Effects on a color image



(a) twirl

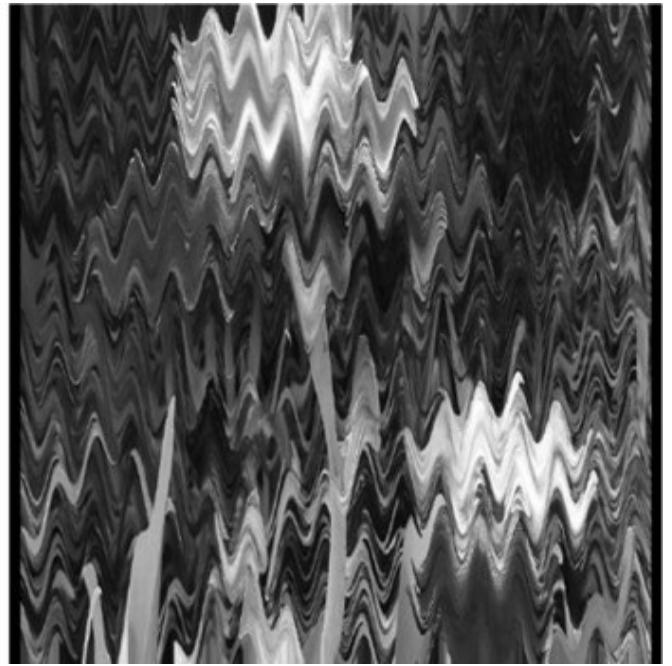
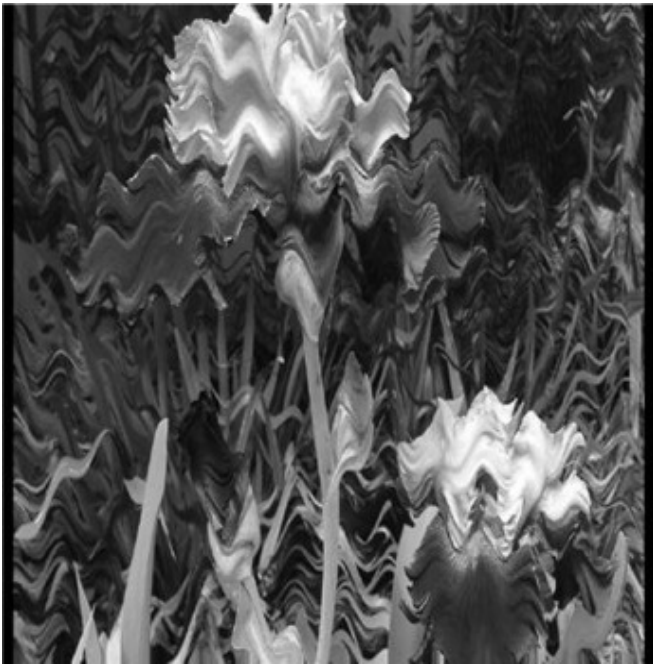


(b) ripple

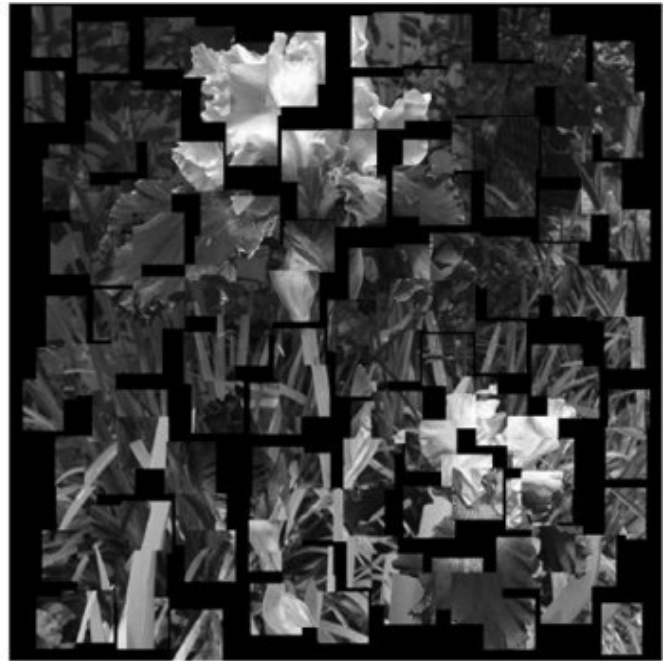
Oil painting is again done by applying the effect to each of the color components separately.

Exercises

1. Experiment with changing the parameters to the effects described in this chapter. For example, see if you can use the pond ripple effect to obtain pictures like this:
-



- 2. Write some functions to implement all the effects given. Include some parameters to change the output.
- 3. Extend your functions so that they can be applied to color images.
- 4. If you can get hold of Holzmann's book, [18]² many other effects are described. For example, the following pictures show the result of a random “tile effect,” where the image is broken up into squares that are then shifted by a random amount.



See if you can write a function to implement this effect, and then extend your function to work with color images.

2

It is now available online, at <http://spinroot.com/pico/>.

Back Matter

Appendix A Introduction to MATLAB and Octave